



T E S Z T P R O J E K T M U N K A

Excalidraw

Függvény alapú lefedettség növelése egy nyílt forráskódú
virtuális whiteboard alkalmazáshoz

Szoftvertesztelés • 2026. tavasz

Balázs-Hegedüs Áron
Cserei Norbert
Nikli Erik

A választott projekt

Excalidraw

Mi ez?

Böngészőben használható whiteboard alkalmazás, kézzel rajzolt stílusú diagramokhoz. Akár virtuális kollaborációra is van lehetőség.

Technológiai stack

- React (a fő frontend keretrendszer)
- TypeScript / TSX (forráskód típusos JS-ben)
- Vitest (teszt framework – ezen nem változtattunk)
- HTML Canvas API (a rajzfelület)

Repók

Eredeti

[github.com/excalidraw/
excalidraw](https://github.com/excalidraw/excalidraw)

Saját fork

[github.com/niklierik/
teszt-beadando-excalidraw](https://github.com/niklierik/teszt-beadando-excalidraw)

Módszertan

Mit és hogyan mértünk

1

Mit mértünk

Függvény alapú lefedettséget (function coverage).

A Vitest beépített coverage moduljával dolgoztunk, ezt nem cseréltük le.

2

Coverage filter

Eredetileg nem volt coverage filter beállítva.

Mi a `packages/**/*.{ts,tsx}` mintára szűkítettük – ez az alkalmazás tényleges forráskódja.

3

Reporter és környezet

Több output formátum: text, json, html, lcovonly.

A tesztek JSdom környezetben futnak (browser DOM emuláció).

A kódbázis felépítése

Monorepo több almodullal

Az Excalidraw kódbázisa egy **monorepo**, ahol több, egymástól külön kezelt almodul (package) található.

A coverage szűrőnk ezekre a *packages* alá eső TS/TSX fájlokra fókuszál.

@excalidraw/common

Közös segédfüggvények, konstansok

@excalidraw/element

Rajz elemek (négyzet, kör, nyíl...)

@excalidraw/excalidraw

A fő React komponens és UI

@excalidraw/math

Geometriai és matematikai műveletek

@excalidraw/utils

Általános utility függvények

@excalidraw/fractional-indexing

Sorrend kezelése a Z-tengelyen

Eredmények

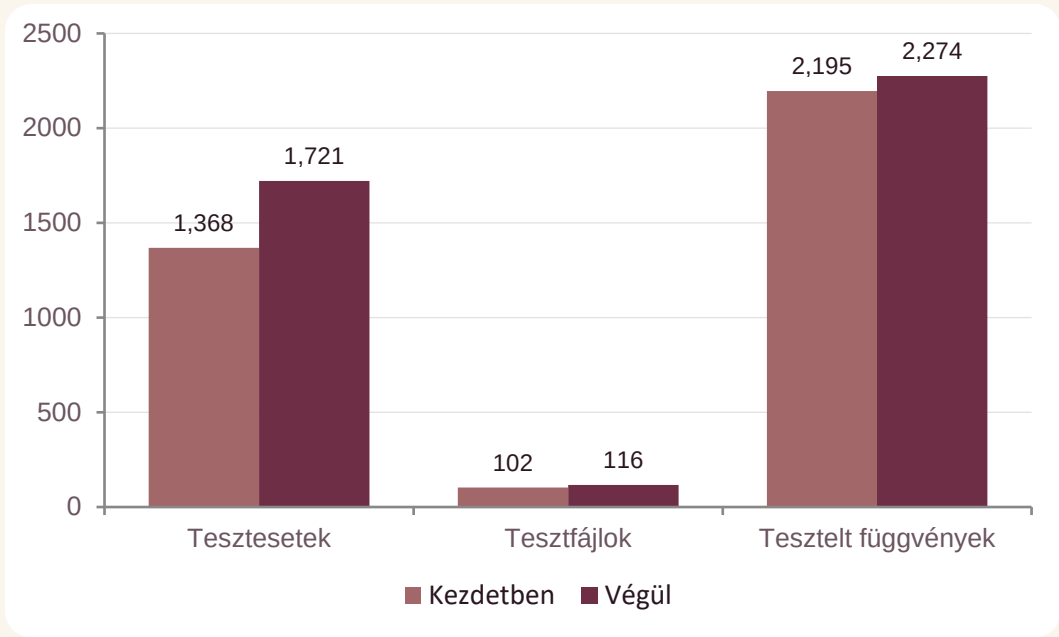
Számokban



Tesztfájlok száma: 102 → 116 (+14) • Az értékek a Vitest coverage report alapján

A növekedés vizuálisan

Kezdő állapot vs. végső állapot



Kulcs adatok

+353

új teszteset

+14

új tesztfájl

+79

új tesztelt függvény

Kihívások

A nehezebben tesztelhető részek

React komponensek

A React komponensek is függvénynek számítanak a coverage-mérésben, ezért ezeket is kellett tesztelni.

A komponenseket virtuális DOM-ra kell rendereli, amihez a React maga ad lehetőséget. Új tesztfájlokat is írtunk ezekhez.

*Példa: **CodeMirrorEditor.test.tsx***

Browser API-k mockolása

A JSdom nem implementálja az összes böngésző API-t, ezeket pótolni kell:

- *vitest-canvas-mock* — Canvas API
- *fake-indexeddb* — tárolás mock
- *matchMedia, FontFace* — kézi mock
- *setPointerCapture* — egér mock
- *font fetch* — fájlrendszerből

Coverage küszöbök

A minőség minimumai

A *vitest.config.mts* fájlban küszöbértékek vannak beállítva. Ha a lefedettség ezek alá esik, a teszt futás sikertelen lesz – ez védi a kódbázist a regressziótól.

Sorok

60%

Ágak

70%

Függvények

63%

Utasítások

60%

A 74,82% függvény lefedettségünk magabiztosan a 63%-os küszöb felett van.

Tapasztalatok

Amit a projektből vittünk magunkkal

Jól lefedett projekteknél nehéz tartalmas tesztet írni

Eleinte több projektet is néztünk (lodash, moment.js), de azok lefedettsége már nagyon magas volt. Az Excalidraw arányosan több, érdemi munkára nyitva álló felülettel rendelkezett.

A coverage szám csak akkor megbízható, ha tudjuk, mit mér

Coverage filter nélkül a generált fájlok (pl. woff2 fontok) is befolyásolnák a számokat. A packages mappára szűrve a mérés a valódi forráskódra fókuszál.

A komponensek tesztelése külön eszközöket igényel

A React komponensekhez virtuális DOM, a Canvas API-hoz canvas mock, az IndexedDB-hez fake-indexeddb kell. A tesztkörnyezet összerakása önmagában is jelentős munka.

Összegzés

- ✓ Sikeresen növeltük a függvény alapú lefedettséget **72,32%-ról 74,82%-ra**
- ✓ 353 új teszteset és 14 új tesztfájl, beleértve nehezebben tesztelhető **React komponenseket**
- ✓ Coverage filter beállítva — a mérés most már a valódi forráskódot fedi le

Köszönjük a figyelmet!